

Omni-Compass C++20 Engine: Source Code

Copyright © 2026 The Omni-Compass LLC. All rights reserved.

The C++20 engine of Omni-Compass: the six-state core, the governor and its allocation laws, the safety shield, the HPA replica law, the closure law, the conveyance law, the nervous system, the compass and its ledger, and the GPU governor's rules, with the command-line tools that run them. Each law is the twin of its Python original and proven equal to it by a parity test (results/SEAL.json).

Version: git commit d76e863315600fd0ea6ac1bdcf89012c3b04ef1c

Date: 2026-09-28

Files: 31

Lines: 1,278

Use of this code is governed by the LICENSE file distributed with it.

Contents

cpp/CMakeLists.txt	28 lines	3
cpp/include/omnicompass/closure.hpp	22 lines	4
cpp/include/omnicompass/compass.hpp	36 lines	5
cpp/include/omnicompass/conveyance.hpp	25 lines	6
cpp/include/omnicompass/core.hpp	38 lines	7
cpp/include/omnicompass/governor.hpp	22 lines	8
cpp/include/omnicompass/gpu_rules.hpp	22 lines	9
cpp/include/omnicompass/hpa.hpp	16 lines	10
cpp/include/omnicompass/nervous_system.hpp	30 lines	11
cpp/include/omnicompass/shield.hpp	13 lines	12
cpp/src/closure.cpp	41 lines	13
cpp/src/compass.cpp	119 lines	14
cpp/src/conveyance.cpp	82 lines	16
cpp/src/core.cpp	40 lines	18
cpp/src/csv.cpp	12 lines	19
cpp/src/governor.cpp	69 lines	20
cpp/src/gpu_rules.cpp	73 lines	21
cpp/src/hpa.cpp	19 lines	22
cpp/src/nervous_system.cpp	71 lines	23
cpp/src/shield.cpp	76 lines	24
cpp/tools/run_closure.cpp	28 lines	25
cpp/tools/run_conveyance.cpp	31 lines	26
cpp/tools/run_fixture.cpp	4 lines	27
cpp/tools/run_governor.cpp	49 lines	28
cpp/tools/run_hpa.cpp	26 lines	29
cpp/tools/run_shield.cpp	42 lines	30
cpp/tools/run_twins.cpp	118 lines	31
cpp/tools/savings.cpp	31 lines	33
cpp/tools/smoke.cpp	29 lines	34
cpp/tools/soak.cpp	47 lines	35
cpp/scripts/compare_results.py	19 lines	36

cpp/CMakeLists.txt

28 lines SHA-256 24a6dd0884a2ed9fd86728e73787465cfaa58509e9cb1e99f41af0c65e078404

```
1 cmake_minimum_required(VERSION 3.16)
2 project(OmniCompass LANGUAGES CXX)
3 set(CMAKE_CXX_STANDARD 20)
4 set(CMAKE_CXX_STANDARD_REQUIRED ON)
5 add_library(omnicompass STATIC src/core.cpp src/csv.cpp src/governor.cpp src/shield.cpp src/hpa.cpp src/closure.
  cpp src/conveyance.cpp src/nervous_system.cpp src/compass.cpp src/gpu_rules.cpp)
6 target_include_directories(omnicompass PUBLIC include)
7 add_executable(oc_run_fixture tools/run_fixture.cpp)
8 target_link_libraries(oc_run_fixture PRIVATE omnicompass)
9 add_executable(oc_governor tools/run_governor.cpp)
10 target_link_libraries(oc_governor PRIVATE omnicompass)
11 add_executable(oc_savings tools/savings.cpp)
12 add_executable(oc_hpa tools/run_hpa.cpp)
13 target_link_libraries(oc_hpa PRIVATE omnicompass)
14 add_executable(oc_shield tools/run_shield.cpp)
15 target_link_libraries(oc_shield PRIVATE omnicompass)
16 add_executable(oc_closure tools/run_closure.cpp)
17 target_link_libraries(oc_closure PRIVATE omnicompass)
18 add_executable(oc_conveyance tools/run_conveyance.cpp)
19 target_link_libraries(oc_conveyance PRIVATE omnicompass)
20 add_executable(oc_twins tools/run_twins.cpp)
21 target_link_libraries(oc_twins PRIVATE omnicompass)
22 add_executable(oc_soak tools/soak.cpp)
23 target_link_libraries(oc_soak PRIVATE omnicompass)
24 add_executable(oc_smoke tools/smoke.cpp)
25 target_link_libraries(oc_smoke PRIVATE omnicompass)
26 enable_testing()
27 add_test(NAME core_smoke COMMAND oc_smoke)
28
```

cpp/include/omnicompass/closure.hpp

22 lines SHA-256 dea67ab6c79ea6fdadf98598caaef18ac14e725bdf1e2890ea921e57061b68c9

```
1 // Closure law (C++ twin of omnicompass/closure.py ClosureLaw / ClosureNodes): same arithmetic, same order of
  operations.
2 #pragma once
3 #include <vector>
4 namespace omnicompass {
5 struct ClosureLaw {
6     double rho_max{0.95}, delta{0.05}; int H_add{8}, H_rel{40};
7     double a{0.5}, b{0.1}, g{0.05}, push_release{0.2}; bool turn{true};
8     double delta_rel{-1.0}, z{0.0}; bool tone{false}; int tone_H{960}; int dwell{0};
9 };
10 class ClosureNodes {
11 public:
12     explicit ClosureNodes(const ClosureLaw& law) : law_(law) {}
13     void observe(double r, double dt = 1.0);
14     double fwd_max(int H) const;
15     int reserve(double c) const;
16     int decide(int n, double c, double push, int n_min, int n_max);
17 private:
18     ClosureLaw law_; bool has_{false}; double L_{0.0}, v_{0.0}, s2_{0.0}; int calm_{0};
19     std::vector<double> hist_;
20 };
21 }
22
```

cpp/include/omnicompass/compass.hpp

36 lines SHA-256 be677eafa23c4bf2bc550737579fd14513b81aa7644102c2c2fab67f0a73e57b

```
1 // The compass (C++ twin of omnicompass/compass.py) and the engine's composite storage, the ledger that closes the
2 // circle (C++ twin of omnicompass/storage.py). Same arithmetic, same order of operations.
3 #pragma once
4 #include "omnicompass/core.hpp"
5 #include <map>
6 #include <optional>
7 #include <string>
8 namespace omnicompass {
9 // storage.py
10 double phi_engine(double S, const Parameters& p); // core.phi: basin potential, equation (5)
11 double stable_S(const Parameters& p); // S*, the stable rest of the axle
12 struct CompositeV { double V{}, V_U{}, V_W{}, V_E{}, V_S{}, V_I{}, V_B{}, S_star{}, B_star{}; };
13 CompositeV composite_V(const State& x, const Parameters& p, int sigma = 1);
14 // compass.py
15 double phi_compass(double S, double alpha_s, double beta_s, double delta);
16 double axle(double alpha_s, double beta_s, double delta);
17 double heading(double e, double rate);
18 int quadrant(double e, double rate);
19 bool inward(double level, double move, double eps = 0.01);
20 struct Reading {
21     double heading_deg{}; int letter{}, point{}, quadrant{}; double e{}, rate{}, axle_rest{}, ledger{};
22     bool has_step{}; double ledger_step{}; bool descent{}, omega_held{}, inward_all{}; int circles_closed{};
23 };
24 class Compass {
25 public:
26     Compass(double E_max = 1.0, double alpha_s = 0.12, double beta_s = 0.10, double delta = 0.5)
27         : E_max_(E_max), alpha_s_(alpha_s), beta_s_(beta_s), delta_(delta) {}
28     Reading read(double E, double S, const std::map<std::string, double>& levels = {},
29                 const std::map<std::string, double>& moves = {}, bool forced = false,
30                 const State* x = nullptr, const Parameters* p = nullptr);
31 private:
32     double E_max_, alpha_s_, beta_s_, delta_;
33     std::optional<double> prev_e_, prev_L_; double scale_{0.0}; int turns_{0}; std::optional<int> last_q_;
34 };
35 }
36
```

cpp/include/omnicompass/conveyance.hpp

25 lines SHA-256 c2c32491b85ec40d236e5204407106018ef782d732834e7d2161953b62c48918

```
1 // Conveyance law (C++ twin of omnicompass/conveyance.py): a conserved budget moved between organs by need.
2 // Same arithmetic, same order of operations: replicator_step (the exact closed form of the replicator flow),
  project
3 // (water-filling onto floors, ceilings and the budget), and Conveyance::step (living band, reserve, flow,
  projection,
4 // continuous release of the reserve). Proofs: docs/CONVEYANCE_LAW.md.
5 #pragma once
6 #include <vector>
7 namespace omnicompass {
8 inline constexpr double BAND_LO = 0.05, BAND_HI = 0.95; // the living band (omnicompass/nervous_system.py BAND)
9 std::vector<double> deficits(const std::vector<double>& a, const std::vector<double>& d, double rho);
10 std::vector<double> replicator_step(const std::vector<double>& a, const std::vector<double>& d, double rho,
    double kappa,
11                                     double dt = 1.0);
12 std::vector<double> project(std::vector<double> a, const std::vector<double>& lo, const std::vector<double>& hi,
13                             double budget);
14 class Conveyance {
15 public:
16     Conveyance(int n, double budget, double rho = 0.8, double kappa = 1.0);
17     std::vector<double> step(const std::vector<double>& need, const std::vector<double>& lo, const std::vector<double>& hi);
18     double budget, rho, kappa;
19     std::vector<double> a;
20 private:
21     std::vector<double> prev_, rate_;
22     bool has_prev_{false};
23 };
24 }
25
```

cpp/include/omnicompass/core.hpp

38 lines SHA-256 a5268199fef7e788940d9fa27278d0d17fffa8b4f3f785b7677eebfc452d775e

```
1  #pragma once
2  #include <array>
3  #include <string>
4  #include <vector>
5  namespace omnicompass {
6  inline constexpr double EPS=1e-12;
7  inline constexpr int MACRO_STEPS=20;
8  inline constexpr int MICRO_STEPS=10;
9  inline constexpr double MACRO_DT=0.1;
10 inline constexpr double MICRO_DT=0.01;
11 inline constexpr double BASIN_TOL=0.10;
12 inline constexpr int CONVEY_WINDOW=5;
13 inline constexpr int CERT_WINDOW=10;
14 inline constexpr double KP=12.0;
15 inline constexpr double U_AUTHORITY=25.0;
16 struct State { double E{},U{},I_U{},S{},B{},B_dot{}; };
17 struct Parameters {
18     double alpha{},beta_int{},beta_ext{},k{},sigma_1{},delta{},gamma_1{},gamma_c{};
19     double lambda_0{},lambda_1{},lambda_2{},c{},E_max{},omega_B{},Q_B{},alpha_s{},beta_s{};
20     double mu{},alpha_E{},alpha_U{},lambda_I{},lambda_U{};
21 };
22 struct InputCase { int run_id{}; State initial{}; Parameters p{}; };
23 struct Result {
24     int run_id{}; State final{}; std::array<double,21> U_hist{};
25     int target_sign{}; int born{}; int convey{}; int convey_start{-1}; int convey_confirm{-1};
26     int cert{}; int cert_start{-1}; int cert_complete{-1}; int max_streak{};
27     double actuator_abs_integral{},actuator_sq_integral{},actuator_peak{}; int actuator_saturated_periods{};
28 };
29 int basin_sign(double U) noexcept;
30 int nearest_basin_sign(double U) noexcept;
31 double v_eff(const State&,const Parameters&,double t) noexcept;
32 State derivatives(const State&,const Parameters&,double t,double u_ctrl=0.0) noexcept;
33 State rk4_step(const State&,const Parameters&,double t,double dt,double u_hold) noexcept;
34 Result simulate(const InputCase&) noexcept;
35 std::vector<InputCase> read_inputs(const std::string& path);
36 void write_results(const std::string& path,const std::vector<Result>& rows);
37 }
38
```

cpp/include/omnicompass/governor.hpp

22 lines SHA-256 42a7a157d21de43e83421ac45b84237294e5ef6978cac290dfde2bcae797640a

```
1 #pragma once
2 // Omni-Compass stack governor: sense -> assimilate -> evolve (u = 0) -> allocate.
3 #include "omnicompass/core.hpp"
4 namespace omnicompass {
5 struct Observation { double q{}, load{}, power{}, thermal{}, network{}, drift{}, stale{}, security{}; int
  conflicts{}; };
6 struct AllocationLaw {
7   double rho0{0.914}, rho_min{0.627}, kI{0.252}, kE{0.297}, kq{0.795}; int up_max{2}, down_max{1};
8   double guard_queue{0.05}, power_ceiling{0.95}, power_ceiling_backlog{1.00}, spread_step{0.05}; bool
  backlog_release{false}, size_at_full_cap{false}; double push_release{0.005}, push_add{-9.0}; double U_gate{0.
  481}, margin{0.006}, cap_min{0.706}, B_cap{0.40}, cap_gain{0.60};
9   double S_rollback{0.55}; double route_network{0.20}; int down_band{4}, down_dwell{4}, down_after_add{4}; int
  min_nodes{8}, max_nodes{40};
10 };
11 struct Directive { int node_delta{}; bool change_permitted{}, rollback_authorized{}, route_shift{};
12   double power_cap{}, demand{}, command{}; State x{}; };
13 class Governor {
14 public:
15   explicit Governor(AllocationLaw law = {}, bool evolve = true);
16   Directive step(const Observation& obs, double current_cap, int nodes);
17   const State& state() const noexcept { return x_; }
18 private:
19   AllocationLaw L_; bool evolve_; State x_; Parameters p_; double t_{0.0}; int surplus_streak_{0}, since_add_{99};
20 };
21 }
22
```

cpp/include/omnicompass/gpu_rules.hpp

22 lines SHA-256 17d1aec370dacc1b9e850f978291a8bd61371578678f68a8c756568f38f25d28

```
1 // GPU governor decision rules (C++ twin of the pure functions in omni_controller/gpu_governor.py): the shield's
2 // limit, the busy gate, the speed lock's step, the baseline lookup and the response-time window. The parts that
  talk to
3 // the device (nvidia-smi) stay in the Python controller; every decision it takes goes through these rules.
4 #pragma once
5 #include <optional>
6 #include <string>
7 #include <vector>
8 namespace omnicompass {
9   struct Limit { int want{}; std::string bound; };
10   Limit shield_limit(double cap, double draw, double start, double min_w, double headroom, double min_share, bool
      slo_clean);
11   struct Gate { double u{}; bool gated{}; };
12   Gate busy_gate(std::optional<double> u_prev, double util, bool gated_prev, double gate, double band);
13   struct LockStep { int want{}; std::string bound; double line{}, aim{}; };
14   LockStep lock_decide(double ratio, double cur, double start, double min_w, double since_last, double speed_gain,
15     double lock_margin, double lock_step, double lock_boost, double lock_floor, double
      lock_hold_s);
16   struct BaselinePoint { double rate{}, mean{}, p95{}, p99{}; };
17   BaselinePoint baseline_at(const std::vector<BaselinePoint>& bins, double rate);
18   struct Row { double t{}, ms{}; bool ok{}; };
19   struct WindowStats { double mean{}, p95{}, p99{}, rate{}; int n{}; };
20   std::optional<WindowStats> window_stats(const std::vector<Row>& rows, double window_s);
21 }
22
```

cpp/include/omnicompass/hpa.hpp

16 lines SHA-256 9fbffe76746981e2cb32a9b3b43b6d366e72eb085d53baafb9dd1a568444ea0

```
1 #pragma once
2 // HorizontalPodAutoscaler replica law (documented behaviour): ratio rule, 10% tolerance, scale-up limited to
3 // max(4 pods, 100%) per period, scale-down to the highest recommendation within the stabilization window.
4 #include <deque>
5 namespace omnicompass {
6 struct HpaParams { double tolerance{0.1}; int window_periods{20}; int up_pods{4}; };
7 class Hpa {
8 public:
9     Hpa(int replicas, int min_rep, int max_rep, HpaParams p = {});
10     int step(double metric, double target);
11     int replicas() const noexcept { return replicas_; }
12 private:
13     int replicas_, min_, max_; HpaParams p_; std::deque<int> recs_;
14 };
15 }
16
```

cpp/include/omnicompass/nervous_system.hpp

30 lines SHA-256 4de0d0370c6635a43009feed13dde064303beb321303fda5dcc0c4d26f7f9995

```
1 // Nervous system (C++ twin of omnicompass/nervous_system.py): one engine state grants authority to every organ.
2 // Same arithmetic, same order of operations. The living band [0.05, 0.95] is applied last to every envelope.
3 #pragma once
4 #include <array>
5 #include <map>
6 #include <string>
7 namespace omnicompass {
8 double stress_equilibrium(double delta, double alpha_s, double beta_s); // omnicompass/closure.py, equation (6)
9 std::array<double, 2> in_band(double lo, double hi);
10 struct NervousInputs {
11     double E{}, U{}, I_U{}, S{}, push{}, push_release{0.2}, U_gate{0.5}, s_eq{1.0};
12     double security_block{0.0}; bool slo_clean{true}; double power_stress{0.0}, thermal{0.0};
13     bool rollback{false}; double stale{0.0}; bool autopilot{true}; bool killed{false};
14 };
15 struct Scalars { double kappa{}, h{}, sigma{}, nu{}, calm{}; };
16 struct Organ { bool expand{}, contract{}; double step{}; std::array<double, 2> envelope{0.0, 0.0}; bool
    has_envelope{false}; };
17 struct Authority {
18     bool execute{}, killed{}, security_hold{};
19     Scalars scalars{};
20     std::map<std::string, Organ> organs; // pods, nodes, cpufreq, gpu, power, routing, cooling
21     bool batch_admit{}, batch_pause{}, rollback_authorized{};
22 };
23 Scalars scalars(const NervousInputs& i);
24 Authority authority(const NervousInputs& i);
25 struct ReleaseGate { bool ok{}; double util_after{}; };
26 ReleaseGate node_release_gate(int n, double per_node_m, double used_m, int pending, bool pods_scaling_up,
27     bool latency_breach_now, double rho, bool node_may_contract, bool senses_live =
    true,
28     bool last_command_landed = true);
29 }
30
```

cpp/include/omnicompass/shield.hpp

13 lines SHA-256 ea047bb2fca4f4997c31d45f3c18596e5c1fc80ffb63fb05249c085627043c61

```
1 #pragma once
2 // Omni-Compass safety shield: invariants I1-I5 enforced on an action set before execution.
3 #include <string>
4 #include <vector>
5 namespace omnicompass {
6 struct Action { std::string kind; double target{}; int direction{}; };
7 struct ShieldLimits { double power_limit{1.00}; int max_node_step{4}; double cap_lo{0.65}, cap_hi{1.00}; };
8 struct ShieldState { int actual_nodes{}; double power_cap{}; double power_stress{}; double security_block{}; int
  min_nodes{}, max_nodes{}; };
9 struct ShieldResult { std::vector<Action> actions; int interventions{}; };
10 ShieldResult enforce(const std::vector<Action>& actions, const ShieldState& s, const ShieldLimits& lim = {});
11 std::vector<std::string> violations(const std::vector<Action>& actions, const ShieldState& s, const ShieldLimits&
  lim = {});
12 }
13
```

cpp/src/closure.cpp

41 lines SHA-256 b4993552a89fa371d45ab614de3ea90ce4ca0ba47404867239ed7cb36489eb14

```
1 #include "omnicompass/closure.hpp"
2 #include <algorithm>
3 #include <cmath>
4 namespace omnicompass {
5 void ClosureNodes::observe(double r, double dt) {
6     if (law_.tone) { hist_.push_back(r); if (static_cast<int>(hist_.size()) > law_.tone_H) hist_.erase(hist_.begin(
7     )); }
8     if (!has_) { L_ = r; has_ = true; return; }
9     const double pred = L_ + v_ * dt, e = r - pred;
10    s2_ = 0.95 * s2_ + 0.05 * e * e;
11    L_ = pred + law_.a * e;
12    v_ = (1.0 - law_.g) * v_ + law_.b * e / dt;
13 }
14 double ClosureNodes::fwd_max(int H) const {
15     if (!has_) return 0.0;
16     double m = L_;
17     for (int tau = 0; tau <= H; ++tau) m = std::max(m, L_ + v_ * tau);
18     return m;
19 }
20 int ClosureNodes::reserve(double c) const {
21     if (hist_.empty()) return 0;
22     const double mx = *std::max_element(hist_.begin(), hist_.end());
23     return static_cast<int>(std::ceil(mx / ((law_.rho_max - law_.delta) * c) - 1e-9));
24 }
25 int ClosureNodes::decide(int n, double c, double push, int n_min, int n_max) {
26     if (!has_) return n;
27     const double peak_add = std::max(fwd_max(law_.H_add), 0.0);
28     if (n <= 0 || peak_add / (n * c) - law_.rho_max > -law_.delta) {
29         calm_ = 0;
30         const int need = static_cast<int>(std::ceil(peak_add / ((law_.rho_max - law_.delta) * c) - 1e-9));
31         return std::min(n_max, std::max({n_min, need, n}));
32     }
33     const double peak_rel = std::max(fwd_max(law_.H_rel), 0.0) + law_.z * std::sqrt(s2_ * std::max(1, law_.H_rel))
34     * law_.a;
35     const double band = law_.delta_rel < 0 ? law_.delta : law_.delta_rel;
36     const bool calm = n - 1 >= n_min && peak_rel / ((n - 1) * c) <= law_.rho_max - band && (!law_.turn || v_ <= 0.
37     0) &&
38         push <= law_.push_release;
39     calm_ = calm ? calm_ + 1 : 0;
40     if (calm && calm_ > law_.dwell) { calm_ = 0; return n - 1; }
41     return n;
42 }
```

cpp/src/compass.cpp

119 lines SHA-256 34b85ae49e1beace8132e10bd3ef8c0dca114fe0f9cb97a1f109db5ac099d7cc

```
1 // C++ twin of omnicompass/compass.py and omnicompass/storage.py. Every line here has its Python line.
2 #include "omnicompass/compass.hpp"
3 #include "omnicompass/conveyance.hpp"
4 #include <algorithm>
5 #include <cmath>
6 #include <stdexcept>
7 namespace omnicompass {
8
9 static constexpr double PI = 3.14159265358979323846;
10
11 // ---- storage.py -----
12 double phi_engine(double S, const Parameters& p) { return 0.5 * p.alpha_s * S * S + 0.25 * p.beta_s * std::pow(S,
13 3.0) - p.delta * S; }
14 double stable_S(const Parameters& p) {
15     const double a = 0.75 * p.beta_s, b = p.alpha_s, c = -p.delta;
16     if (std::abs(a) <= EPS) return p.delta / std::max(p.alpha_s, EPS);
17     const double disc = std::max(0.0, b * b - 4.0 * a * c);
18     for (double r : {(-b + std::sqrt(disc)) / (2.0 * a), (-b - std::sqrt(disc)) / (2.0 * a)})
19         if (-p.alpha_s - 1.5 * p.beta_s * r < 0.0) return r;
20     throw std::runtime_error("no stable S rest point");
21 }
22
23 static void bath_P(const Parameters& p, double& p11, double& p12, double& p22) {
24     const double w = std::max(std::abs(p.omega_B), EPS);
25     const double damp = std::max(w / std::max(std::abs(p.Q_B), EPS), EPS);
26     p12 = 1.0 / (2.0 * w * w); p22 = (p12 + 0.5) / damp; p11 = damp * p12 + w * w * p22;
27 }
28
29 CompositeV composite_V(const State& x, const Parameters& p, int sigma) {
30     const double sig = sigma >= 0 ? 1.0 : -1.0;
31     const double alpha_s = std::max(p.alpha_s, EPS), beta_s = std::max(p.beta_s, EPS);
32     double p11, p12, p22; bath_P(p, p11, p12, p22);
33     const double g = p.gamma_c * p.delta;
34     const double pg0 = p12 * g, pg1 = p22 * g;
35     const double m_floor = alpha_s / 2.0;
36     const double a_U = 1.0, b_W = 1.0 / (2.0 * std::max(p.mu, EPS)), c_E = 1.0 / std::max(p.alpha_E, EPS);
37     const double w_I = 1.0 / std::max(p.lambda_I, EPS), w_S = (std::pow(pg0, 2.0) + std::pow(pg1, 2.0) + 0.25) / (
38 m_floor * m_floor), d_B = 1.0;
39     (void)beta_s;
40     const double s_star = stable_S(p);
41     const double ob = std::max(std::abs(p.omega_B), EPS);
42     const double b_star = p.gamma_c * p.delta / std::pow(ob, 2.0) * s_star;
43     const double z0 = x.B - b_star, z1 = x.B_dot;
44     CompositeV v;
45     v.V_U = 0.5 * a_U * std::pow(x.U - sig, 2.0);
46     v.V_W = b_W * 0.25 * p.mu * std::pow(x.U * x.U - 1.0, 2.0);
47     v.V_E = 0.5 * c_E * std::pow(x.E, 2.0);
48     v.V_S = w_S * (phi_engine(x.S, p) - phi_engine(s_star, p));
49     v.V_I = 0.5 * w_I * std::pow(x.I_U, 2.0);
50     v.V_B = 0.5 * d_B * (p11 * z0 * z0 + 2 * p12 * z0 * z1 + p22 * z1 * z1);
51     v.V = v.V_U + v.V_W + v.V_E + v.V_S + v.V_I + v.V_B;
52     v.S_star = s_star; v.B_star = b_star;
53     return v;
54 }
55 // ---- compass.py -----
56 double phi_compass(double S, double alpha_s, double beta_s, double delta) {
57     return alpha_s * S * S / 2.0 + beta_s * std::pow(S, 3.0) / 4.0 - delta * S;
58 }
59
60 double axle(double alpha_s, double beta_s, double delta) {
61     const double q = 0.75 * beta_s;
62     if (q <= 0) return alpha_s > 0 ? delta / alpha_s : 0.0;
63     return (-alpha_s + std::sqrt(alpha_s * alpha_s + 4.0 * q * delta)) / (2.0 * q);
64 }
65
66 double heading(double e, double rate) {
67     if (std::abs(e) < 1e-12 && std::abs(rate) < 1e-12) return 0.0;
68     double h = std::fmod(std::atan2(e, rate) * (180.0 / PI), 360.0);
69     if (h < 0) h += 360.0;
70     return h;
71 }
72
73 int quadrant(double e, double rate) {
74     if (e > 0) return rate > 0 ? 1 : 2;
75     return rate < 0 ? 3 : 4;
76 }
77
78 bool inward(double level, double move, double eps) {
79     if (level >= BAND_HI - eps && move > 0) return false;
```

```

80     if (level <= BAND_LO + eps && move < 0) return false;
81     return true;
82 }
83
84 static double pymod(double a, double m) { double r = std::fmod(a, m); return r < 0 ? r + m : r; }
85
86 Reading Compass::read(double E, double S, const std::map<std::string, double>& levels,
87                     const std::map<std::string, double>& moves, bool forced, const State* x, const Parameters*
88 p) {
89     const double e = E / std::max(E_max_, 1e-12);
90     const double raw = prev_e_ ? e - *prev_e_ : 0.0;
91     scale_ = scale_ == 0.0 ? raw * raw : 0.9 * scale_ + 0.1 * raw * raw;
92     const double rate = scale_ > 1e-18 ? raw / std::sqrt(scale_) : 0.0;
93     Reading r;
94     const double h = heading(e, rate);
95     const int q = quadrant(e, rate);
96     if (last_q_ && *last_q_ == 4 && q == 1) ++turns_;
97     r.point = int(pymod(h + 22.5, 360.0) / 45.0);
98     r.letter = int(pymod(h + 7.5, 360.0) / 15.0);
99     double s_star = axle(alpha_s_, beta_s_, delta_), L;
100     if (x && p) {
101         const CompositeV cv = composite_V(*x, *p, 1);
102         L = cv.V; s_star = cv.S_star;
103     } else {
104         L = 0.5 * e * e + phi_compass(S, alpha_s_, beta_s_, delta_) - phi_compass(s_star, alpha_s_, beta_s_, delta_);
105     }
106     bool held = true, inw = true;
107     for (const auto& [k, v] : levels) {
108         if (!(BAND_LO - 1e-9 <= v && v <= BAND_HI + 1e-9)) held = false;
109         const auto it = moves.find(k);
110         if (!inward(v, it == moves.end() ? 0.0 : it->second)) inw = false;
111     }
112     r.heading_deg = h; r.quadrant = q; r.e = e; r.rate = rate; r.axle_rest = s_star; r.ledger = L;
113     r.has_step = prev_L_.has_value(); r.ledger_step = r.has_step ? L - *prev_L_ : 0.0;
114     r.descent = r.has_step && (r.ledger_step <= 1e-9 || forced);
115     r.omega_held = held; r.inward_all = inw; r.circles_closed = turns_;
116     prev_e_ = e; prev_L_ = L; last_q_ = q;
117     return r;
118 }
119

```

cpp/src/conveyance.cpp

82 lines SHA-256 948b267e1593aa8acd46f9d36fb4b6bea5d04bf4122242679792eb2696eb618

```
1 // C++ twin of omnicompass/conveyance.py. Read the two side by side: every line here has its Python line.
2 #include "omnicompass/conveyance.hpp"
3 #include <algorithm>
4 #include <cmath>
5 namespace omnicompass {
6
7 static double sum(const std::vector<double>& v) { double s = 0.0; for (double x : v) s += x; return s; }
8
9 std::vector<double> deficits(const std::vector<double>& a, const std::vector<double>& d, double rho) {
10     std::vector<double> e(a.size());
11     for (size_t i = 0; i < a.size(); ++i) e[i] = d[i] / (rho * std::max(a[i], 1e-12)) - 1.0;
12     return e;
13 }
14
15 std::vector<double> replicator_step(const std::vector<double>& a, const std::vector<double>& d, double rho,
16     double kappa, double dt) {
17     const double A = sum(a), D = sum(d);
18     if (A <= 0 || D <= 0) return a;
19     const double k = std::exp(-kappa * D * dt / (rho * A));
20     std::vector<double> out(a.size());
21     for (size_t i = 0; i < a.size(); ++i) out[i] = A * d[i] / D + (a[i] - A * d[i] / D) * k;
22     return out;
23 }
24
25 std::vector<double> project(std::vector<double> a, const std::vector<double>& lo, const std::vector<double>& hi,
26     double budget) {
27     const size_t n = a.size();
28     for (size_t i = 0; i < n; ++i) a[i] = std::min(std::max(a[i], lo[i]), hi[i]);
29     for (size_t round = 0; round < n + 1; ++round) {
30         const double tot = sum(a);
31         if (tot <= budget + 1e-9) return a;
32         std::vector<char> free(n, 0);
33         double share = 0.0;
34         for (size_t i = 0; i < n; ++i) if (a[i] > lo[i] + 1e-12) { free[i] = 1; share += a[i] - lo[i]; }
35         const double over = tot - budget;
36         if (share <= 0) return a;
37         std::vector<double> b(n);
38         for (size_t i = 0; i < n; ++i) b[i] = free[i] ? std::max(lo[i], a[i] - over * (a[i] - lo[i]) / share) : a[i];
39         a = b;
40     }
41     return a;
42 }
43
44 Conveyance::Conveyance(int n, double budget_, double rho_, double kappa_)
45     : budget(budget_), rho(rho_), kappa(kappa_), a(n, budget_ / n), rate_(n, 0.0) {}
46
47 std::vector<double> Conveyance::step(const std::vector<double>& need_in, const std::vector<double>& lo_in,
48     const std::vector<double>& hi_in) {
49     const size_t n = need_in.size();
50     // the living band: every organ keeps at least 5% of its range (idle, never off) and never takes more than 95%
51     std::vector<double> lo(n), hi(n), need(n);
52     for (size_t i = 0; i < n; ++i) lo[i] = std::max(lo_in[i], BAND_LO * hi_in[i]);
53     for (size_t i = 0; i < n; ++i) hi[i] = std::max(lo[i], BAND_HI * hi_in[i]);
54     for (size_t i = 0; i < n; ++i) need[i] = std::max(need_in[i], 1e-9);
55     if (has_prev_) // rate tracker for the reserve (the dual-bath rate channel)
56         for (size_t i = 0; i < n; ++i) rate_[i] = 0.5 * rate_[i] + 0.5 * (need[i] - prev_[i]);
57     prev_ = need; has_prev_ = true;
58     double rise = 0.0; for (double r : rate_) rise += std::max(0.0, r);
59     const double reserve = std::min(0.5 * budget, rise);
60     std::vector<double> want(n);
61     for (size_t i = 0; i < n; ++i) want[i] = std::max(std::min(hi[i], need[i] / rho), lo[i]); // usable at rho,
62     // floored
63     const double spend = std::max(sum(lo), std::min(budget - reserve, sum(want))); // never spend
64     // unneeded budget
65     std::vector<double> dneed(n);
66     for (size_t i = 0; i < n; ++i) dneed[i] = want[i] * rho;
67     std::vector<double> x = replicator_step(a, dneed, rho, kappa);
68     const double sx = std::max(sum(x), 1e-12);
69     for (size_t i = 0; i < n; ++i) x[i] = x[i] * spend / sx;
70     std::vector<double> ceil(n);
71     const bool enough = sum(want) <= spend;
72     for (size_t i = 0; i < n; ++i) ceil[i] = enough ? std::max(lo[i], std::min(hi[i], want[i])) : hi[i];
73     x = project(x, lo, ceil, spend);
74     // surge (Ch. 30): release the reserve continuously to organs still in deficit, up to their ceiling
75     const double left = budget - sum(x);
76     std::vector<double> short_(n); double tot = 0.0;
77     for (size_t i = 0; i < n; ++i) { short_[i] = std::max(0.0, std::min(hi[i], need[i] / rho) - x[i]); tot +=
78         short_[i]; }
79     if (tot > 0 && left > 0)
80         for (size_t i = 0; i < n; ++i) x[i] = x[i] + std::min(short_[i], left * short_[i] / tot);
81     a = x;
82 }
```

```
79     return a;  
80 }  
81 }  
82
```

cpp/src/core.cpp

40 lines SHA-256 d7deead6f47d54d5b68e08b98db5e15ab54c7fbac8e7b9c24ee64cb1538a8c5c

```
1 #include "omnicompass/core.hpp"
2 #include <algorithm>
3 #include <cmath>
4 namespace omnicompass {
5 static State add_scaled(const State&a,const State&b,double s) noexcept{return {a.E+s*b.E,a.U+s*b.U,a.I_U+s*b.I_U,
6   a.S+s*b.S,a.B+s*b.B,a.B_dot+s*b.B_dot};}
7 static State combine(const State&s,const State&k1,const State&k2,const State&k3,const State&k4,double h) noexcept{
8   return {s.E+h*(k1.E+2*k2.E+2*k3.E+k4.E)/6.0,s.U+h*(k1.U+2*k2.U+2*k3.U+k4.U)/6.0,s.I_U+h*(k1.I_U+2*k2.I_U+2*k3.
9     I_U+k4.I_U)/6.0,s.S+h*(k1.S+2*k2.S+2*k3.S+k4.S)/6.0,s.B+h*(k1.B+2*k2.B+2*k3.B+k4.B)/6.0,s.B_dot+h*(k1.
10     B_dot+2*k2.B_dot+2*k3.B_dot+k4.B_dot)/6.0};}
11 int basin_sign(double U) noexcept {if(std::abs(U-1.0)<=BASIN_TOL)return 1;if(std::abs(U+1.0)<=BASIN_TOL)return -1;
12   return 0;}
13 int nearest_basin_sign(double U) noexcept {return std::abs(U-1.0)<=std::abs(U+1.0)?1:-1;}
14 double v_eff(const State&s,const Parameters&p,double t) noexcept {double z=p.lambda_0+p.lambda_1*(s.U-0.5)+p.
15   lambda_2*s.S;return std::cos(0.5*p.omega_B*t)*(p.c*std::tanh(z));}
16 State derivatives(const State&s,const Parameters&p,double t,double u) noexcept {
17   double v=v_eff(s,p,t); double dE=-p.alpha_E*s.E+p.beta_int+p.beta_ext+v; double emax=std::max(p.E_max,1e-6);
18   double dU=p.mu*s.U*(1.0-s.U*s.U)-dE/emax-p.lambda_U*s.U+u;
19   double dI=(1.0-s.U)-p.sigma_1*s.E-p.delta*s.S-p.lambda_I*s.I_U;
20   double dS=p.delta-p.alpha_s*s.S-0.75*p.beta_s*s.S*s.S;
21   double dB=s.B_dot;
22   double dBd=p.gamma_c*p.delta*s.S-(p.omega_B/std::max(p.Q_B,EPS))*s.B_dot-p.omega_B*p.omega_B*s.B;
23   return {dE,dU,dI,dS,dB,dBd};
24 }
25 State rk4_step(const State&s,const Parameters&p,double t,double h,double u) noexcept {State k1=derivatives(s,p,t,
26   u);State k2=derivatives(add_scaled(s,k1,0.5*h),p,t+0.5*h,u);State k3=derivatives(add_scaled(s,k2,0.5*h),p,t+0.
27   5*h,u);State k4=derivatives(add_scaled(s,k3,h),p,t+h,u);return combine(s,k1,k2,k3,k4,h);}
28 Result simulate(const InputCase&in) noexcept {
29   Result r{}; r.run_id=in.run_id; r.U_hist[0]=in.initial.U; r.target_sign=nearest_basin_sign(in.initial.U); r.
30     born=basin_sign(in.initial.U)!=0;
31   int sign0=basin_sign(in.initial.U), streak=r.born?1:0, streak_sign=r.born?sign0:0, cert_streak=streak,
32     cert_sign=streak_sign; r.max_streak=streak;
33   State s=in.initial; double t=0.0;
34   for(int macro=1;macro<=MACRO_STEPS;++macro){
35     for(int mi=0;mi<MICRO_STEPS;++mi){
36       double drift=derivatives(s,in.p,t,0.0).U; double raw=-drift+KP*(double(r.target_sign)-s.U); double u=std::max(-
37         U_AUTHORITY,std::min(U_AUTHORITY,raw));
38       r.actuator_abs_integral+=std::abs(u)*MICRO_DT; r.actuator_sq_integral+=u*u*MICRO_DT; r.actuator_peak=std::max(
39         r.actuator_peak,std::abs(u)); if(std::abs(raw)>U_AUTHORITY+EPS)++r.actuator_saturated_periods;
40       s=rk4_step(s,in.p,t,MICRO_DT,u); t+=MICRO_DT;
41     }
42     r.U_hist[macro]=s.U; int bs=basin_sign(s.U); bool inside=bs!=0;
43     if(inside){ if(bs==streak_sign)++streak; else{streak_sign=bs;streak=1;} } else {streak_sign=0;streak=0;}
44     if(inside){ if(bs==cert_sign)++cert_streak; else{cert_sign=bs;cert_streak=1;} } else {cert_sign=0;cert_streak=0;}
45   }
46   r.max_streak=std::max({r.max_streak,streak,cert_streak});
47   if(!r.convey && streak>=CONVEY_WINDOW){r.convey=1;r.convey_start=macro-CONVEY_WINDOW+1;r.convey_confirm=macro;}
48   if(!r.cert && cert_streak>=CERT_WINDOW){r.cert=1;r.cert_start=macro-CERT_WINDOW+1;r.cert_complete=macro;}
49 }
50 r.final=s; return r;
51 }
52 }
```

cpp/src/csv.cpp

12 lines SHA-256 841147a0c7a19593885ed38ea2954f96d1baeaad54202fa57d34ac419da72c2c

```
1 #include "omnicompass/core.hpp"
2 #include <fstream>
3 #include <iomanip>
4 #include <sstream>
5 #include <stdexcept>
6 #include <unordered_map>
7 namespace omnicompass {
8 static std::vector<std::string> split(const std::string&s){std::vector<std::string>v;std::stringstream ss(s);std::
    string x;while(std::getline(ss,x,',')){if(!x.empty() && x.back()=='\r')x.pop_back();v.push_back(x);}return v;}
9 std::vector<InputCase> read_inputs(const std::string&path){std::ifstream f(path);if(!f)throw std::runtime_error(
    "cannot open "+path);std::string line;if(!std::getline(f,line))throw std::runtime_error("empty csv");auto
    h=split(line);std::unordered_map<std::string,size_t>idx;for(size_t i=0;i<h.size();++i)idx[h[i]]=i;auto d=[&](
    const std::vector<std::string>&r,const char*n){return std::stoi(r.at(idx.at(n)));};std::vector<InputCase>out;
    while(std::getline(f,line)){if(line.empty())continue;auto r=split(line);InputCase x{};x.run_id=int(d(r,
    "run_id"));x.initial={d(r,"E0"),d(r,"U0"),d(r,"I0"),d(r,"S0"),d(r,"B0"),d(r,"Bdot0")};x.p={d(r,"alpha"),d(r,
    "beta_int"),d(r,"beta_ext"),d(r,"k"),d(r,"sigma_1"),d(r,"delta"),d(r,"gamma_1"),d(r,"gamma_c"),d(r,"lambda_0"),
    d(r,"lambda_1"),d(r,"lambda_2"),d(r,"c"),d(r,"E_max"),d(r,"omega_B"),d(r,"Q_B"),d(r,"alpha_s"),d(r,"beta_s"),d(
    r,"mu"),d(r,"alpha_E"),d(r,"alpha_U"),d(r,"lambda_I"),d(r,"lambda_U")};out.push_back(x);}return out;}
10 void write_results(const std::string&path,const std::vector<Result>&rows){std::ofstream f(path);if(!f)throw std::
    runtime_error("cannot write "+path);f<<"run_id,E,U,I_U,S,B,B_dot,target_sign,born,convey,convey_start,
    convey_confirm,cert,cert_start,cert_complete,max_streak,actuator_abs_integral,actuator_sq_integral,
    actuator_peak,actuator_saturated_periods";for(int i=0;i<=20;++i)f<<"U_ "<<i;f<<"\n";f<<std::setprecision(17);
    for(auto&r:rows){f<<r.run_id<<','<<r.final.E<<','<<r.final.U<<','<<r.final.I_U<<','<<r.final.S<<','<<r.final.
    B<<','<<r.final.B_dot<<','<<r.target_sign<<','<<r.born<<','<<r.convey<<','<<r.convey_start<<','<<r.
    convey_confirm<<','<<r.cert<<','<<r.cert_start<<','<<r.cert_complete<<','<<r.max_streak<<','<<r.
    actuator_abs_integral<<','<<r.actuator_sq_integral<<','<<r.actuator_peak<<','<<r.actuator_saturated_periods;for(
    double u:r.U_hist)f<<','<<u;f<<'\n';}}
11 }
12 }
```

cpp/src/governor.cpp

69 lines SHA-256 8fcedb561ba6101881429b6309bbb2a806599b90b48233f00685ad5c7f654cbb

```
1 #include "omnicompass/governor.hpp"
2 #include <algorithm>
3 #include <cmath>
4 namespace omnicompass {
5 static double cl(double v, double lo, double hi) { return v < lo ? lo : (v > hi ? hi : v); }
6
7 Governor::Governor(AllocationLaw law, bool evolve) : L_(law), evolve_(evolve) {
8     x_ = {0.12, 0.82, 0.0, 0.0, 0.0, 0.0};
9     p_ = {4.2, 0.0, 0.0, 1.7, 0.38, 0.705, 1.35, 1.0, -0.40, 1.55, 1.15, 1.0, 2.0, 1.0, 2.5, 0.12, 0.10, 3.782, 3.
10         487, 4.2, 0.719, 0.0};
11 }
12 Directive Governor::step(const Observation& ob, double current_cap, int nodes) {
13     const double q = cl(ob.q, 0, 2), load = cl(ob.load, 0, 2), pw = cl(ob.power, 0, 1.5), th = cl(ob.thermal, 0, 1.
14         5);
15     const double nw = cl(ob.network, 0, 1.5), dr = cl(ob.drift, 0, 1.5), st = cl(ob.stale, 0, 1), sec = cl(ob.
16         security, 0, 1);
17     const double cf = cl(ob.conflicts / 4.0, 0, 1);
18     const double e_obs = cl(0.25*q + 0.18*std::max(0.0, load-0.85) + 0.16*pw + 0.13*th + 0.10*nw + 0.08*dr + 0.
19         06*cf + 0.04*st, 0, 1.5);
20     const double u_obs = cl(1.0 - (0.27*q + 0.18*pw + 0.16*th + 0.12*nw + 0.12*dr + 0.10*cf + 0.05*st), 0, 1);
21     const double s_obs = cl(0.36*th + 0.28*pw + 0.18*nw + 0.10*sec + 0.08*st - 0.20*q, -1, 1);
22     const double b_obs = cl(0.52*pw + 0.26*th + 0.22*dr, -1, 1.5);
23     const double i_obs = cl(q + dr + cf, 0, 2);
24     const double a = 0.339;
25     p_.beta_int = cl(0.58*q + 0.42*std::max(0.0, load-0.75), 0, 1);
26     p_.beta_ext = cl(0.33*pw + 0.27*th + 0.20*nw + 0.12*st + 0.08*sec, 0, 1);
27     const double U_before = x_.U;
28     State xa{(1-a)*x_.E + a*e_obs, (1-a)*x_.U + a*u_obs, (1-a)*x_.I_U + a*i_obs,
29         (1-a)*x_.S + a*s_obs, (1-a)*x_.B + a*b_obs, (1-a)*x_.B_dot + a*(b_obs - x_.B)};
30     if (evolve_) {
31         double t = t_;
32         for (int i = 0; i < MICRO_STEPS; ++i) { xa = rk4_step(xa, p_, t, MICRO_DT, 0.0); t += MICRO_DT; }
33     }
34     x_ = xa; t_ += MACRO_DT;
35     Directive d{}; d.x = x_;
36     const int n = nodes;
37     const double rho = cl(L_.rho0 - L_.kI*x_.I_U - L_.kE*x_.E, L_.rho_min, L_.rho0);
38     const double load_sized = L_.size_at_full_cap ? load * current_cap : load;
39     const long n_req = static_cast<long>(std::ceil(n * load_sized / rho)) + static_cast<long>(std::ceil(L_.kq * q *
40         n));
41     long delta = std::max<long>(-L_.down_max, std::min<long>(L_.up_max, n_req - n));
42     const double drift_U = derivatives(x_, p_, t_, 0.0).U;
43     const double u_push = std::max(-U_AUTHORITY, std::min(U_AUTHORITY, -drift_U + KP * (1.0 - x_.U)));
44     const bool converged = u_push / U_AUTHORITY <= L_.push_release;
45     if (delta < 0 && !converged) delta = 0;
46     if (delta > 0 && q < L_.guard_queue && u_push / U_AUTHORITY < L_.push_add) delta = 0;
47     if (delta < 0 && (q >= L_.guard_queue || n_req > n - L_.down_band)) delta = 0;
48     surplus_streak_ = delta < 0 ? surplus_streak_ + 1 : 0;
49     if (delta < 0 && (surplus_streak_ < L_.down_dwell || since_add_ < L_.down_after_add)) delta = 0;
50     const double ceiling = q >= L_.guard_queue ? L_.power_ceiling_backlog : L_.power_ceiling;
51     if (delta > 0 && (pw >= ceiling || n >= L_.max_nodes)) delta = 0;
52     if (sec > 0.5) delta = std::min<long>(delta, 0);
53     since_add_ = delta > 0 ? 0 : since_add_ + 1;
54     if (delta < 0) surplus_streak_ = 0;
55     const bool at_envelope = pw >= ceiling;
56     double cap;
57     if (q < L_.guard_queue && x_.U >= L_.U_gate) cap = cl(current_cap * load * (1.0 + L_.margin), L_.cap_min, 1.0);
58     else if (q >= L_.guard_queue && L_.spread_step > 0.0)
59         cap = at_envelope ? cl(current_cap - L_.spread_step, L_.cap_min, 1.0) : (L_.backlog_release ? 1.0 : (n < L_.
60             max_nodes ? current_cap : 1.0));
61     else cap = 1.0;
62     cap = std::min(cap, cl(1.0 - L_.cap_gain * std::max(0.0, x_.B - L_.B_cap), L_.cap_min, 1.0));
63     if (delta > 0) cap = std::max(cap, current_cap);
64     d.command = cl(0.5 * (v_eff(x_, p_, 0.0) / std::max(p_.c, 1e-9) + 1.0), 0, 1);
65     d.demand = rho;
66     d.node_delta = static_cast<int>(delta);
67     d.change_permitted = x_.U >= L_.U_gate && sec <= 0.5;
68     d.power_cap = cap;
69     d.rollback_authorized = x_.S >= L_.S_rollback || x_.U < U_before || sec > 0.5;
70     d.route_shift = nw > L_.route_network;
71     return d;
72 }
```

cpp/src/gpu_rules.cpp

73 lines SHA-256 66072cd30a7ab82fe937d5e44138a67038134e932c2883406f8fac26b40ecd76

```
1 // C++ twin of the GPU governor's rules (omni_controller/gpu_governor.py: shield_limit, busy_gate, lock_decide,
2 // baseline_at, window_stats). Every line here has its Python line.
3 #include "omnicompass/gpu_rules.hpp"
4 #include <algorithm>
5 #include <cmath>
6 namespace omnicompass {
7
8 Limit shield_limit(double cap, double draw, double start, double min_w, double headroom, double min_share, bool
9 slo_clean) {
10     const double share_floor = min_share * start;
11     const double floor_ = std::max({draw * (1.0 + headroom), min_w, share_floor});
12     const double engine_w = std::floor(cap * start);
13     const int want = int(std::min(start, std::max(engine_w, std::ceil(floor_))));
14     std::string bound;
15     if (!slo_clean) bound = "slo_or_blind_reflex";
16     else if (want >= start && engine_w >= start) bound = "start_ceiling";
17     else if (std::ceil(floor_) > engine_w && floor_ == share_floor) bound = "share_floor";
18     else if (std::ceil(floor_) > engine_w && floor_ > min_w) bound = "draw_headroom_floor";
19     else if (std::ceil(floor_) > engine_w) bound = "device_minimum";
20     else bound = "engine";
21     return {want, bound};
22 }
23
24 Gate busy_gate(std::optional<double> u_prev, double util, bool gated_prev, double gate, double band) {
25     const double u = 0.5 * (u_prev ? *u_prev : util) + 0.5 * util;
26     return {u, u >= gate || (gated_prev && u >= gate - band)};
27 }
28
29 LockStep lock_decide(double ratio, double cur, double start, double min_w, double since_last, double speed_gain,
30 double lock_margin, double lock_step, double lock_boost, double lock_floor, double
31 lock_hold_s) {
32     const double line = 1.0 - speed_gain;
33     const double aim = line * (1.0 - lock_margin);
34     const double floor_ = std::max(min_w, lock_floor * start);
35     double want; std::string bound;
36     if (ratio > line) { want = start; bound = "speed_lock_release"; }
37     else if (ratio < aim && since_last >= lock_hold_s) {
38         const double mult = std::min(lock_boost, std::max(1.0, (aim - ratio) / std::max(1e-6, line * lock_margin)));
39         want = std::max(floor_, cur - mult * lock_step * start); bound = "speed_lock_spend";
40     } else { want = cur; bound = "speed_lock_hold"; }
41     return {int(std::min(start, std::max(floor_, want))), bound, line, aim};
42 }
43
44 BaselinePoint baseline_at(const std::vector<BaselinePoint>& bins, double rate) {
45     if (rate <= bins.front().rate) return bins.front();
46     if (rate >= bins.back().rate) return bins.back();
47     for (size_t i = 0; i + 1 < bins.size(); ++i) {
48         const BaselinePoint& lo = bins[i]; const BaselinePoint& hi = bins[i + 1];
49         if (lo.rate <= rate && rate <= hi.rate) {
50             const double f = (rate - lo.rate) / (hi.rate - lo.rate);
51             return {rate, lo.mean + f * (hi.mean - lo.mean), lo.p95 + f * (hi.p95 - lo.p95), lo.p99 + f * (hi.p99 - lo.p99)};
52         }
53     }
54     return bins.back();
55 }
56
57 std::optional<WindowStats> window_stats(const std::vector<Row>& rows, double window_s) {
58     if (rows.empty()) return std::nullopt;
59     const double t_end = rows.back().t;
60     std::vector<double> ms;
61     for (const Row& r : rows) if (r.t >= t_end - window_s && r.ok) ms.push_back(r.ms);
62     if (ms.size() < 20) return std::nullopt;
63     std::sort(ms.begin(), ms.end());
64     const double span = std::min(window_s, std::max(1.0, t_end - rows.front().t));
65     double s = 0.0; for (double m : ms) s += m;
66     const size_t n = ms.size();
67     WindowStats w;
68     w.mean = s / double(n);
69     w.p95 = ms[std::min(n - 1, size_t(0.95 * double(n)))];
70     w.p99 = ms[std::min(n - 1, size_t(0.99 * double(n)))];
71     w.rate = double(n) / span; w.n = int(n);
72     return w;
73 }
```

cpp/src/hpa.cpp

19 lines SHA-256 91e9b487afe6e8db4f3b355516a0ec85d2730073f5c3e7b628c6e3a155535eaa

```
1 #include "omnicompass/hpa.hpp"
2 #include <algorithm>
3 #include <cmath>
4 namespace omnicompass {
5 Hpa::Hpa(int replicas, int min_rep, int max_rep, HpaParams p) : replicas_(replicas), min_(min_rep), max_(max_rep),
  p_(p) {}
6 int Hpa::step(double metric, double target) {
7     const double ratio = metric / std::max(target, 1e-9);
8     const int cur = replicas_;
9     int rec = cur;
10    if (std::abs(ratio - 1.0) > p_.tolerance)
11        rec = std::max(min_, std::min(max_, static_cast<int>(std::ceil(static_cast<double>(cur) * ratio))));
12    recs_.push_back(rec);
13    while (static_cast<int>(recs_.size()) > p_.window_periods) recs_.pop_front();
14    const int next = rec > cur ? std::min(rec, cur + std::max(p_.up_pods, cur)) : *std::max_element(recs_.begin(),
  recs_.end());
15    replicas_ = std::max(min_, std::min(max_, next));
16    return replicas_;
17 }
18 }
19
```

cpp/src/nervous_system.cpp

71 lines SHA-256 1ce2ad02713d9b6f351c17a4e8dfa56c851a50f134c2639b07a30005209bfbd3

```
1 // C++ twin of omnicompass/nervous_system.py. Read the two side by side: every line here has its Python line.
2 #include "omnicompass/nervous_system.hpp"
3 #include "omnicompass/conveyance.hpp"
4 #include <algorithm>
5 #include <cmath>
6 #include <limits>
7 namespace omnicompass {
8
9 static double clamp(double x, double lo, double hi) { return x < lo ? lo : (x > hi ? hi : x); }
10
11 double stress_equilibrium(double delta, double alpha_s, double beta_s) {
12     const double qa = 0.75 * beta_s;
13     return qa > 0 ? (-alpha_s + std::sqrt(alpha_s * alpha_s + 4.0 * qa * delta)) / (2.0 * qa) : delta / alpha_s;
14 }
15
16 std::array<double, 2> in_band(double lo, double hi) {
17     const double h = std::min(BAND_HI, std::max(BAND_LO, hi));
18     return {std::min(h, std::max(BAND_LO, lo)), h};
19 }
20
21 Scalars scalars(const NervousInputs& i) {
22     Scalars s;
23     s.kappa = i.push <= i.push_release ? 1.0 : i.push_release / std::max(i.push, 1e-12);
24     s.h = clamp((i.U - i.U_gate) / std::max(1.0 - i.U_gate, 1e-12), 0.0, 1.0);
25     s.sigma = i.s_eq > 0 ? i.S / i.s_eq : 0.0;
26     s.nu = std::max(0.0, i.I_U);
27     const double calm = s.kappa * s.h * (1.0 - clamp(s.sigma - 1.0, 0.0, 1.0)) * (1.0 - clamp(s.nu, 0.0, 1.0));
28     s.calm = clamp(calm, 0.0, 1.0);
29     return s;
30 }
31
32 Authority authority(const NervousInputs& i) {
33     Authority a;
34     if (i.killed) { a.execute = false; a.killed = true; return a; }
35     a.scalars = scalars(i); const double calm = a.scalars.calm;
36     const bool sec = i.security_block > 0.5;
37     const bool hot = i.thermal >= 0.96;
38     const double excess = clamp(i.thermal - 0.96, 0.0, 1.0);
39     const bool seeing = i.stale <= 0.0; // never give capacity back on a blind sense; expanding stays allowed
40     static const std::pair<const char*, double> THETA[] = {{"pods", 0.5}, {"nodes", 0.7}, {"cpufreq", 0.3},
41                                                         {"gpu", 0.3}, {"power", 0.3}, {"routing", 0.5}};
42     for (const auto& [o, th] : THETA) {
43         const bool ok = calm >= th && i.slo_clean && seeing;
44         a.organs[o] = Organ{!sec, ok, ok ? calm : 0.0};
45     }
46     const double cf_lo = 1.0 - 0.35 * calm, gp_lo = 1.0 - 0.30 * calm;
47     const double cf_hi = hot ? 1.0 - 0.35 * excess : 1.0, gp_hi = hot ? 1.0 - 0.35 * excess : 1.0;
48     a.organs["cpufreq"].envelope = in_band(std::min(cf_lo, cf_hi), cf_hi); a.organs["cpufreq"].has_envelope = true;
49     a.organs["gpu"].envelope = in_band(std::min(gp_lo, gp_hi), gp_hi); a.organs["gpu"].has_envelope = true;
50     a.organs["power"].envelope = in_band(0.65, 1.0); a.organs["power"].has_envelope = true;
51     // routing is an amount moved, not a level: it may move nothing, never more than the band's top
52     a.organs["routing"].envelope = {0.0, !sec ? std::min(BAND_HI, 0.5 * calm) : 0.0}; a.organs["routing"].
53     has_envelope = true;
54     a.organs["cooling"] = Organ{true, calm >= 0.3 && seeing, calm, {18.0, 18.0 + 9.0 * calm}, true};
55     a.batch_admit = (!sec) && seeing && calm >= 0.5 && i.power_stress < 0.9;
56     a.batch_pause = a.scalars.sigma > 1.0 || i.power_stress >= 0.95 || hot;
57     a.rollback_authorized = i.rollback || sec;
58     a.execute = i.autopilot; a.killed = false; a.security_hold = sec;
59     return a;
60 }
61
62 ReleaseGate node_release_gate(int n, double per_node_m, double used_m, int pending, bool pods_scaling_up,
63                               bool latency_breach_now, double rho, bool node_may_contract, bool senses_live,
64                               bool last_command_landed) {
65     ReleaseGate r;
66     r.util_after = n > 1 ? used_m / std::max((n - 1) * per_node_m, 1e-9) : std::numeric_limits<double>::infinity();
67     r.ok = n > 1 && pending == 0 && !pods_scaling_up && !latency_breach_now && r.util_after <= rho &&
68     node_may_contract &&
69     senses_live && last_command_landed;
70     return r;
71 }
```

cpp/src/shield.cpp

76 lines SHA-256 bfa324a7b5c33323b5c5b05f85ccb3f2bfdc35e0636740fb61ac513324fa50c8

```
1 #include "omnicompass/shield.hpp"
2 #include <algorithm>
3 #include <cmath>
4 namespace omnicompass {
5 static bool is_expand(const std::string& k) { return k == "nodes" || k == "terraform_plan" || k == "rollout" || k
   == "replicas"; }
6 static bool is_infra(const std::string& k) { return k == "nodes" || k == "terraform_plan" || k == "rollout"; }
7
8 std::vector<std::string> violations(const std::vector<Action>& acts, const ShieldState& s, const ShieldLimits&
   lim) {
9     std::vector<std::string> v;
10    const int n = std::max(1, s.actual_nodes);
11    const bool sec = s.security_block > 0.5;
12    for (const auto& a : acts) {
13        if (sec && a.direction > 0 && is_infra(a.kind)) v.push_back("I1");
14        if (a.kind == "nodes") {
15            const int t = static_cast<int>(a.target);
16            const bool inside = s.min_nodes <= t && t <= s.max_nodes;
17            const bool toward = (n > s.max_nodes && s.max_nodes <= t && t < n) || (n < s.min_nodes && n < t && t <= s.
   min_nodes);
18            if (!inside && !toward) v.push_back("I2");
19            const int add = t - n;
20            if (std::abs(add) > lim.max_node_step) v.push_back("I5");
21            if (add > 0 && s.power_stress * (n + add) / n > lim.power_limit) v.push_back("I4");
22        }
23        if (a.kind == "power_cap" && !(lim.cap_lo <= a.target && a.target <= lim.cap_hi)) v.push_back("I2");
24    }
25    bool up = false, tighten = false;
26    for (const auto& a : acts) {
27        up = up || (a.direction > 0 && is_expand(a.kind));
28        tighten = tighten || (a.kind == "power_cap" && a.target < s.power_cap - 1e-9);
29    }
30    if (up && tighten) v.push_back("I3");
31    return v;
32 }
33
34 ShieldResult enforce(const std::vector<Action>& acts, const ShieldState& s, const ShieldLimits& lim) {
35     ShieldResult r;
36     const int n = std::max(1, s.actual_nodes);
37     const bool sec = s.security_block > 0.5;
38     const double ps = s.power_stress;
39     for (Action a : acts) {
40         if (sec && a.direction > 0 && is_infra(a.kind)) { ++r.interventions; continue; }
41         if (a.kind == "nodes") {
42             const int r0 = static_cast<int>(a.target);
43             const bool toward = (n > s.max_nodes && s.max_nodes <= r0 && r0 < n) || (n < s.min_nodes && n < r0 && r0 <=
   s.min_nodes);
44             int t = toward ? r0 : std::max(s.min_nodes, std::min(s.max_nodes, r0)); // minimal intervention
45             t = std::max(n - lim.max_node_step, std::min(n + lim.max_node_step, t));
46             if (t > n && ps > 0.0) {
47                 const double k_max_d = std::min(1.0e6, (lim.power_limit / ps) * n - n + 1e-9);
48                 const int k_max = static_cast<int>(k_max_d);
49                 t = std::min(t, n + std::max(0, k_max));
50             }
51             if (sec && t > n) { ++r.interventions; continue; } // I1 after clamping
52             if (t != static_cast<int>(a.target)) ++r.interventions;
53             if (t == n) continue;
54             a.target = t; a.direction = t > n ? 1 : -1;
55         }
56         if (a.kind == "power_cap") {
57             const double c = std::max(lim.cap_lo, std::min(lim.cap_hi, a.target));
58             if (c != a.target) ++r.interventions;
59             a.target = c;
60         }
61         r.actions.push_back(a);
62     }
63     bool up = false;
64     for (const auto& a : r.actions) up = up || (a.direction > 0 && is_expand(a.kind));
65     if (up) {
66         std::vector<Action> kept;
67         for (const auto& a : r.actions) {
68             if (a.kind == "power_cap" && a.target < s.power_cap - 1e-9) { ++r.interventions; continue; }
69             kept.push_back(a);
70         }
71         r.actions = kept;
72     }
73     return r;
74 }
75 }
76 }
```

cpp/tools/run_closure.cpp

28 lines SHA-256 d9566c0a0a8f74ae6a091b063a61a2d5b9d9dfd00d2946e32c0988a253f322e7

```
1 // oc_closure LAW.csv TRACE.csv OUT.csv : law parameters (one row), then per tick "r,n,c,push,n_min,n_max";
2 // writes per tick "target,reserve" from the C++ closure law.
3 #include "omnicompass/closure.hpp"
4 #include <cstdio>
5 #include <fstream>
6 #include <sstream>
7 #include <string>
8 using namespace omnicompass;
9 int main(int argc, char** argv) {
10     if (argc < 4) { std::fprintf(stderr, "usage: oc_closure LAW.csv TRACE.csv OUT.csv\n"); return 2; }
11     std::ifstream lf(argv[1]); std::string line; std::getline(lf, line); std::getline(lf, line);
12     std::stringstream ls(line); std::string f; std::vector<double> v;
13     while (std::getline(ls, f, ',')) v.push_back(std::stod(f));
14     ClosureLaw L; L.rho_max = v[0]; L.delta = v[1]; L.H_add = int(v[2]); L.H_rel = int(v[3]); L.a = v[4]; L.b = v[
5]; L.g = v[6];
15     L.push_release = v[7]; L.turn = v[8] != 0; L.delta_rel = v[9]; L.z = v[10]; L.tone = v[11] != 0; L.tone_H = int(
v[12]); L.dwell = int(v[13]);
16     ClosureNodes nodes(L);
17     std::ifstream tf(argv[2]); std::ofstream out(argv[3]); std::getline(tf, line); out << "target,reserve\n";
18     char buf[64];
19     while (std::getline(tf, line)) {
20         std::stringstream ss(line); std::vector<double> x;
21         while (std::getline(ss, f, ',')) x.push_back(std::stod(f));
22         nodes.observe(x[0]);
23         const int t = nodes.decide(int(x[1]), x[2], x[3], int(x[4]), int(x[5]));
24         std::snprintf(buf, sizeof buf, "%d,%d\n", t, nodes.reserve(x[2])); out << buf;
25     }
26     return 0;
27 }
28
```

cpp/tools/run_conveyance.cpp

31 lines SHA-256 88482e90eb5a4068ee55f103038589e9aca0dffb4d7458db9fd94a301021e75d

```
1 // oc_conveyance IN.csv OUT.csv : first row "n,budget,rho,kappa"; then per step 3n values "need..., lo..., hi...";
2 // writes per step the n allocations from the C++ conveyance law (17 significant digits).
3 #include "omnicompass/conveyance.hpp"
4 #include <cstdio>
5 #include <fstream>
6 #include <sstream>
7 #include <string>
8 using namespace omnicompass;
9 static std::vector<double> row(const std::string& line) {
10     std::stringstream ss(line); std::string f; std::vector<double> v;
11     while (std::getline(ss, f, ',')) v.push_back(std::stod(f));
12     return v;
13 }
14 int main(int argc, char** argv) {
15     if (argc < 3) { std::fprintf(stderr, "usage: oc_conveyance IN.csv OUT.csv\n"); return 2; }
16     std::ifstream in(argv[1]); std::ofstream out(argv[2]); std::string line;
17     std::getline(in, line); const std::vector<double> h = row(line);
18     const int n = int(h[0]);
19     Conveyance cv(n, h[1], h[2], h[3]);
20     char buf[40];
21     while (std::getline(in, line)) {
22         if (line.empty()) continue;
23         const std::vector<double> v = row(line);
24         std::vector<double> need(v.begin(), v.begin() + n), lo(v.begin() + n, v.begin() + 2 * n), hi(v.begin() + 2 *
n, v.begin() + 3 * n);
25         const std::vector<double> a = cv.step(need, lo, hi);
26         for (int i = 0; i < n; ++i) { std::snprintf(buf, sizeof buf, i ? ",%.17g" : "%.17g", a[i]); out << buf; }
27         out << "\n";
28     }
29     return 0;
30 }
31
```

cpp/tools/run_fixture.cpp

4 lines SHA-256 c4931db605bb24c077e5740fa0f0ed631766ff568ec487caec5dd2313165dd38

```
1 #include "omniconpass/core.hpp"
2 #include <iostream>
3 int main(int argc, char**argv){if(argc!=3){std::cerr<<"usage: oc_run_fixture INPUT.csv OUTPUT.csv\n";return 2;}try{
    auto in=omniconpass::read_inputs(argv[1]);std::vector<omniconpass::Result>out;out.reserve(in.size());for(auto&x:
    in)out.push_back(omniconpass::simulate(x));omniconpass::write_results(argv[2],out);std::cout<<"processed " <<out.
    size()<<" current-engine fixtures\n";return 0;}catch(const std::exception&e){std::cerr<<e.what()<<"\n";return 1;
    }}
4
```

cpp/tools/run_governor.cpp

49 lines SHA-256 18b4c7ff3a92c52f29ee3533dd3eed8200bf2120fb6c9d2c2a9055c76f1fcb61

```
1 // Reads an observation stream (CSV), runs one Governor per scenario, writes directives (CSV).
2 #include "omnicompass/governor.hpp"
3 #include <fstream>
4 #include <iomanip>
5 #include <iostream>
6 #include <map>
7 #include <sstream>
8 #include <string>
9 #include <vector>
10 using namespace omnicompass;
11 static std::vector<std::string> split(const std::string& s) { std::vector<std::string> v; std::stringstream ss(s);
12     std::string x;
13     while (std::getline(ss, x, ',')) { if (!x.empty() && x.back() == '\r') x.pop_back(); v.push_back(x); } return v;
14 }
15 int main(int argc, char** argv) {
16     if (argc != 3 && argc != 4) { std::cerr << "usage: oc_governor OBS.csv OUT.csv [power_protect|throughput]\n";
17     return 2; }
18     AllocationLaw law{};
19     if (argc == 4 && (std::string(argv[3]) == "throughput" || std::string(argv[3]).rfind("fleet", 0) == 0)) {
20         law.backlog_release = true; law.power_ceiling = 9.0; law.power_ceiling_backlog = 9.0; law.spread_step = 0.0;
21         law.B_cap = 9.0; law.size_at_full_cap = true;
22         law.kI = 0.05; law.kE = 0.40; law.cap_min = 0.65; law.down_band = 1; law.down_dwell = 10; law.down_after_add
23         = 3;
24         law.kq = 0.30; law.U_gate = 0.45; law.push_release = 0.01;
25     }
26     if (argc == 4 && std::string(argv[3]).rfind("fleet", 0) == 0) {
27         law.rho0 = 0.829; law.rho_min = 0.769; law.kI = 0.072; law.kE = 0.487; law.kq = 0.635; law.down_band = 3; law.
28         down_dwell = 1; law.down_after_add = 5; law.push_release = 0.053; law.cap_min = 0.695; law.margin = 0.131; law.
29         U_gate = 0.85; law.up_max = 8;
30     } if (argc == 4 && std::string(argv[3]) == "fleet_balanced") { law.push_release = 0.0098; law.push_add = 0.
31     0098; }
32     if (argc == 4 && std::string(argv[3]) == "fleet_wear") { law.push_release = 0.0035; law.push_add = 0.0035; }
33
34     std::ifstream f(argv[1]); std::ofstream o(argv[2]); std::string line; std::getline(f, line);
35     auto h = split(line); std::map<std::string, size_t> ix; for (size_t i = 0; i < h.size(); ++i) ix[h[i]] = i;
36     o << "scenario_id,step,node_delta,change_permitted,power_cap,rollback_authorized,route_shift,demand,command,E,U,
37     I,U,S,B\n";
38     o << std::setprecision(17);
39     std::map<int, Governor> gov; long n = 0;
40     while (std::getline(f, line)) {
41         if (line.empty()) continue;
42         auto r = split(line);
43         auto d = [&](const char* k) { return std::stod(r.at(ix.at(k))); };
44         int sid = static_cast<int>(d("scenario_id"));
45         Observation ob{d("q"), d("load"), d("power"), d("thermal"), d("network"), d("drift"), d("stale"), d(
46         "security"),
47         static_cast<int>(d("conflicts"))};
48         auto it = gov.find(sid); if (it == gov.end()) it = gov.emplace(sid, Governor{law}).first;
49         Directive dv = it->second.step(ob, d("current_cap"), static_cast<int>(d("nodes")));
50         o << sid << ',' << r.at(ix.at("step")) << ',' << dv.node_delta << ',' << dv.change_permitted << ',' << dv.
51         power_cap << ','
52         << dv.rollback_authorized << ',' << dv.route_shift << ',' << dv.demand << ',' << dv.command << ','
53         << dv.x.E << ',' << dv.x.U << ',' << dv.x.I_U << ',' << dv.x.S << ',' << dv.x.B << '\n';
54         ++n;
55     }
56     std::cout << "processed " << n << " governor steps\n"; return 0;
57 }
```

cpp/tools/run_hpa.cpp

26 lines SHA-256 7c2675ba84dda153a84ba1a1cc986b838560cbfe27e51ff12091e85c9588c738

```
1 // Reads HPA step streams (CSV: stream,replicas0,min,max,metric,target), one Hpa per stream; writes replicas per
  step.
2 #include "omnicompass/hpa.hpp"
3 #include <fstream>
4 #include <iostream>
5 #include <map>
6 #include <sstream>
7 #include <string>
8 #include <vector>
9 using namespace omnicompass;
10 int main(int argc, char** argv) {
11     if (argc != 3) { std::cerr << "usage: oc_hpa STEPS.csv OUT.csv\n"; return 2; }
12     std::ifstream f(argv[1]); std::ofstream o(argv[2]); std::string line; std::getline(f, line);
13     std::map<std::string, Hpa> h; long n = 0;
14     o << "stream,replicas\n";
15     while (std::getline(f, line)) {
16         if (line.empty()) continue;
17         std::vector<std::string> c; std::stringstream ss(line); std::string x;
18         while (std::getline(ss, x, ',')) c.push_back(x);
19         auto it = h.find(c[0]);
20         if (it == h.end()) it = h.emplace(c[0], Hpa(std::stoi(c[1]), std::stoi(c[2]), std::stoi(c[3]))).first;
21         o << c[0] << ',' << it->second.step(std::stod(c[4]), std::stod(c[5])) << '\n'; ++n;
22     }
23     std::cout << "processed " << n << " HPA steps\n";
24     return 0;
25 }
26
```

cpp/tools/run_shield.cpp

42 lines SHA-256 a7edc9a4982ed5e45206b622bad7ef65ce8fe8feb37ec96175200596a108bb8f

```
1 // Reads shield cases (CSV), applies enforce() and violations(), writes results (CSV).
2 #include "omnicompass/shield.hpp"
3 #include <cstdio>
4 #include <fstream>
5 #include <iostream>
6 #include <sstream>
7 #include <string>
8 #include <vector>
9 using namespace omnicompass;
10 static std::vector<std::string> split(const std::string& s, char d) {
11     std::vector<std::string> v; std::stringstream ss(s); std::string x;
12     while (std::getline(ss, x, d)) v.push_back(x);
13     return v;
14 }
15 int main(int argc, char** argv) {
16     if (argc != 3) { std::cerr << "usage: oc_shield CASES.csv OUT.csv\n"; return 2; }
17     std::ifstream f(argv[1]); std::ofstream o(argv[2]); std::string line; std::getline(f, line);
18     o << "case,interventions,enforced,violations\n";
19     long n = 0;
20     while (std::getline(f, line)) {
21         if (line.empty()) continue;
22         auto c = split(line, ',');
23         ShieldState s{std::stoi(c[1]), std::stod(c[2]), std::stod(c[3]), std::stod(c[4]), std::stoi(c[5]), std::stoi(
24             c[6])};
25         ShieldLimits lim{}; lim.power_limit = std::stod(c[7]);
26         std::vector<Action> acts;
27         if (c.size() > 8 && !c[8].empty())
28             for (const auto& a : split(c[8], ',')) { auto p = split(a, ':'); acts.push_back({p[0], std::stod(p[1]), std:
29                 :stoi(p[2])}); }
30         auto r = enforce(acts, s, lim);
31         auto v = violations(acts, s, lim);
32         o << c[0] << ',' << r.interventions << ',';
33         for (size_t i = 0; i < r.actions.size(); ++i) {
34             char buf[64]; std::snprintf(buf, sizeof buf, "%.17g", r.actions[i].target);
35             o << (i ? ";" : "") << r.actions[i].kind << ':' << buf << ':' << r.actions[i].direction;
36         }
37         o << ',';
38         for (size_t i = 0; i < v.size(); ++i) o << (i ? ";" : "") << v[i];
39         o << '\n'; ++n;
40     }
41     std::cout << "processed " << n << " shield cases\n";
42     return 0;
43 }
```

cpp/tools/run_twins.cpp

118 lines SHA-256 60a4e43e1c21d50ffeeefa44f85c867a67e728052b3a6f49677c0ade23f17a849

```
1 // oc_twins MODE IN.csv OUT.csv : feeds recorded inputs to a C++ twin and writes its outputs (17 significant
  digits),
2 // for the parity tests (tests/test_cpp_twins_parity.py). Modes:
3 // nervous rows E,U,I_U,S,push_release,U_gate,s_eq,security_block,slo_clean,power_stress,thermal,
  rollback,stale,autopilot,killed
4 // gate rows n,per_node_m,used_m,pending,scaling_up,breach,rho,may_contract,senses_live,landed
5 // compass first row E_max,alpha_s,beta_s,delta,use_state; rows E,S,l1,m1,l2,m2,forced[E,U,I_U,S,B,B_dot,
  alpha_s,beta_s,delta,omega_B,Q_B,gamma_c,mu,alpha_E,lambda_I]
6 // shield rows cap,draw,start,min_w,headroom,min_share,slo_clean
7 // busy rows util,gate,band (one sequence; the state carries over)
8 // lock rows ratio,cur,start,min_w,since_last,speed_gain,lock_margin,lock_step,lock_boost,lock_floor,
  lock_hold_s
9 // window first row window_s; rows t,ms,ok (one window over all rows)
10 // baseline first row k (bins); k rows rate,mean,p95,p99; then rows rate
11 #include "omnicompass/compass.hpp"
12 #include "omnicompass/gpu_rules.hpp"
13 #include "omnicompass/nervous_system.hpp"
14 #include <cstdio>
15 #include <fstream>
16 #include <sstream>
17 #include <string>
18 #include <vector>
19 using namespace omnicompass;
20 static std::vector<double> row(const std::string& line) {
21     std::stringstream ss(line); std::string f; std::vector<double> v;
22     while (std::getline(ss, f, ',')) v.push_back(std::stod(f));
23     return v;
24 }
25 struct Out {
26     std::ofstream& o; bool first{true};
27     void num(double x) { char b[40]; std::snprintf(b, sizeof b, first ? "%.17g" : ",%.17g", x); o << b; first =
  false; }
28     void str(const std::string& s) { o << (first ? "" : ",") << s; first = false; }
29     void end() { o << "\n"; first = true; }
30 };
31 int main(int argc, char** argv) {
32     if (argc < 4) { std::fprintf(stderr, "usage: oc_twins MODE IN.csv OUT.csv\n"); return 2; }
33     const std::string mode = argv[1];
34     std::ifstream in(argv[2]); std::ofstream of(argv[3]); Out out{of}; std::string line;
35     if (mode == "nervous") {
36         while (std::getline(in, line)) {
37             if (line.empty()) continue;
38             const auto v = row(line);
39             NervousInputs i{v[0], v[1], v[2], v[3], v[4], v[5], v[6], v[7], v[8], v[9] != 0, v[10], v[11], v[12] != 0,
  v[13], v[14] != 0, v[15] != 0};
40             const Authority a = authority(i);
41             out.num(a.execute); out.num(a.killed);
42             if (!a.killed) {
43                 out.num(a.security_hold); out.num(a.scalars.kappa); out.num(a.scalars.h); out.num(a.scalars.sigma); out.
  num(a.scalars.nu); out.num(a.scalars.calm);
44                 for (const char* o : {"pods", "nodes", "cpufreq", "gpu", "power", "routing"}) {
45                     const Organ& g = a.organs.at(o); out.num(g.expand); out.num(g.contract); out.num(g.step);
46                 }
47                 for (const char* o : {"cpufreq", "gpu", "power", "routing"}) { out.num(a.organs.at(o).envelope[0]); out.
  num(a.organs.at(o).envelope[1]); }
48                 const Organ& c = a.organs.at("cooling"); out.num(c.contract); out.num(c.step); out.num(c.envelope[0]);
  out.num(c.envelope[1]);
49                 out.num(a.batch_admit); out.num(a.batch_pause); out.num(a.rollback_authorized);
50                 out.num(stress_equilibrium(v[3] + 0.5, 0.12, 0.10));
51             }
52             out.end();
53         }
54     } else if (mode == "gate") {
55         while (std::getline(in, line)) {
56             if (line.empty()) continue;
57             const auto v = row(line);
58             const ReleaseGate g = node_release_gate(int(v[0]), v[1], v[2], int(v[3]), v[4] != 0, v[5] != 0, v[6], v[7]
  != 0, v[8] != 0, v[9] != 0);
59             out.num(g.ok); out.num(g.util_after); out.end();
60         }
61     } else if (mode == "compass") {
62         std::getline(in, line); const auto h = row(line);
63         Compass c{h[0], h[1], h[2], h[3]}; const bool use_state = h[4] != 0;
64         while (std::getline(in, line)) {
65             if (line.empty()) continue;
66             const auto v = row(line);
67             std::map<std::string, double> lv{{"a", v[2]}, {"b", v[4]}}, mv{{"a", v[3]}, {"b", v[5]}};
68             State x; Parameters p;
69             if (use_state) {
70                 x = State{v[7], v[8], v[9], v[10], v[11], v[12]};
71                 p.alpha_s = v[13]; p.beta_s = v[14]; p.delta = v[15]; p.omega_B = v[16]; p.Q_B = v[17]; p.gamma_c = v[
  18];
72                 p.mu = v[19]; p.alpha_E = v[20]; p.lambda_I = v[21];
```

```

73     }
74     const Reading r = c.read(v[0], v[1], lv, mv, v[6] != 0, use_state ? &x : nullptr, use_state ? &p :
nullptr);
75     out.num(r.heading_deg); out.num(r.letter); out.num(r.point); out.num(r.quadrant); out.num(r.e); out.num(r.
rate);
76     out.num(r.axle_rest); out.num(r.ledger); out.num(r.has_step); out.num(r.ledger_step); out.num(r.descent);
77     out.num(r.omega_held); out.num(r.inward_all); out.num(r.circles_closed); out.end();
78     }
79     } else if (mode == "shield") {
80         while (std::getline(in, line)) {
81             if (line.empty()) continue;
82             const auto v = row(line);
83             const Limit l = shield_limit(v[0], v[1], v[2], v[3], v[4], v[5], v[6] != 0);
84             out.num(l.want); out.str(l.bound); out.end();
85         }
86     } else if (mode == "busy") {
87         std::optional<double> u; bool gated = false;
88         while (std::getline(in, line)) {
89             if (line.empty()) continue;
90             const auto v = row(line);
91             const Gate g = busy_gate(u, v[0], gated, v[1], v[2]); u = g.u; gated = g.gated;
92             out.num(g.u); out.num(g.gated); out.end();
93         }
94     } else if (mode == "lock") {
95         while (std::getline(in, line)) {
96             if (line.empty()) continue;
97             const auto v = row(line);
98             const LockStep s = lock_decide(v[0], v[1], v[2], v[3], v[4], v[5], v[6], v[7], v[8], v[9], v[10]);
99             out.num(s.want); out.str(s.bound); out.num(s.line); out.num(s.aim); out.end();
100         }
101     } else if (mode == "window") {
102         std::getline(in, line); const double w = row(line)[0]; std::vector<Row> rows;
103         while (std::getline(in, line)) { if (line.empty()) continue; const auto v = row(line); rows.push_back({v[0],
v[1], v[2] != 0}); }
104         const auto s = window_stats(rows, w);
105         if (!s) { out.str("none"); out.end(); }
106         else { out.num(s->mean); out.num(s->p95); out.num(s->p99); out.num(s->rate); out.num(s->n); out.end(); }
107     } else if (mode == "baseline") {
108         std::getline(in, line); const int k = int(row(line)[0]); std::vector<BaselinePoint> bins;
109         for (int i = 0; i < k; ++i) { std::getline(in, line); const auto v = row(line); bins.push_back({v[0], v[1],
v[2], v[3]}); }
110         while (std::getline(in, line)) {
111             if (line.empty()) continue;
112             const BaselinePoint b = baseline_at(bins, row(line)[0]);
113             out.num(b.mean); out.num(b.p95); out.num(b.p99); out.end();
114         }
115     } else { std::fprintf(stderr, "unknown mode %s\n", mode.c_str()); return 2; }
116     return 0;
117 }
118

```

cpp/tools/savings.cpp

31 lines SHA-256 a1f0f72285d3c35510678ae44e41238e2a3d448a6809d10314b3a854415ea7e7

```
1 // Savings projector: measured fractional energy reduction (from the benchmark) x a declared fleet profile.
2 // Output is a projection, not a measurement. Input CSV columns:
3 // profile,vessel,baseline,nodes,node_avg_kw,pue,usd_per_kwh,kg_co2_per_kwh,reduction_low,reduction_mid,
  reduction_high
4 #include <cstdio>
5 #include <fstream>
6 #include <iostream>
7 #include <sstream>
8 #include <string>
9 #include <vector>
10 int main(int argc, char** argv) {
11     if (argc != 3) { std::cerr << "usage: oc_savings INPUTS.csv OUT.csv\n"; return 2; }
12     std::ifstream f(argv[1]); std::ofstream o(argv[2]); std::string line; std::getline(f, line);
13     o << "profile,vessel,baseline,baseline_mwh_per_year,saved_mwh_low,saved_mwh_mid,saved_mwh_high,"
14         "saved_usd_low,saved_usd_mid,saved_usd_high,saved_tco2_low,saved_tco2_mid,saved_tco2_high\n";
15     while (std::getline(f, line)) {
16         if (line.empty()) continue;
17         std::vector<std::string> c; std::stringstream ss(line); std::string x;
18         while (std::getline(ss, x, ',')) c.push_back(x);
19         const double nodes = std::stod(c[3]), kw = std::stod(c[4]), pue = std::stod(c[5]), usd = std::stod(c[6]), co2
= std::stod(c[7]);
20         const double base_mwh = nodes * kw * pue * 8760.0 / 1000.0;
21         char buf[512];
22         std::snprintf(buf, sizeof buf, "%s,%s,%s,%.17g", c[0].c_str(), c[1].c_str(), c[2].c_str(), base_mwh);
23         o << buf;
24         for (int k = 0; k < 3; ++k) { std::snprintf(buf, sizeof buf, ",%.17g", base_mwh * std::stod(c[8 + k])); o <<
buf; }
25         for (int k = 0; k < 3; ++k) { std::snprintf(buf, sizeof buf, ",%.17g", base_mwh * 1000.0 * usd * std::stod(c[
8 + k])); o << buf; }
26         for (int k = 0; k < 3; ++k) { std::snprintf(buf, sizeof buf, ",%.17g", base_mwh * co2 * std::stod(c[8 + k]));
o << buf; }
27         o << '\n';
28     }
29     return 0;
30 }
31
```

cpp/tools/smoke.cpp

29 lines SHA-256 69d47e3fa386eb7ea5aaffaef75715c1ca0c585bee81b5a96403f6856ca16e50

```
1 // Governor contract checks: bounds, security, determinism, both modes.
2 #include "omnicompass/governor.hpp"
3 #include <cmath>
4 #include <cstdio>
5 using namespace omnicompass;
6 static int fails = 0;
7 static void expect(bool c, const char* what) { if (!c) { std::printf("FAIL %s\n", what); ++fails; } }
8 int main() {
9     AllocationLaw tp{}; tp.backlog_release = true; tp.power_ceiling = 9.0; tp.power_ceiling_backlog = 9.0; tp.
    spread_step = 0.0;
10    tp.B_cap = 9.0; tp.size_at_full_cap = true; tp.kI = 0.05; tp.kE = 0.40; tp.cap_min = 0.65; tp.down_band = 1; tp.
    down_dwell = 10;
11    tp.down_after_add = 3; tp.kq = 0.30; tp.U_gate = 0.45; tp.push_release = 0.01;
12    for (const AllocationLaw& law : {AllocationLaw{}, tp}) {
13        Governor g{law}, h{law};
14        for (int i = 0; i < 2000; ++i) {
15            const double s = 0.5 + 0.5 * std::sin(0.037 * i);
16            Observation o{0.4 * s, 0.4 + 0.9 * s, 0.5 + 0.6 * s, 0.3 + 0.7 * s, 0.2 * s, 0.1 * s, 0.0, (i % 97 < 5) ? 1.
    0 : 0.0, i % 5};
17            const int n = 8 + (i % 33);
18            Directive d = g.step(o, 0.9, n), e = h.step(o, 0.9, n);
19            expect(d.node_delta >= -law.down_max && d.node_delta <= law.up_max, "node delta within bounds");
20            expect(d.power_cap >= law.cap_min - 1e-12 && d.power_cap <= 1.0 + 1e-12, "power cap within bounds");
21            expect(!(o.security > 0.5) || d.node_delta <= 0, "no capacity addition during security block");
22            expect(std::isfinite(d.x.E) && std::isfinite(d.x.U) && std::isfinite(d.x.I_U) && std::isfinite(d.x.S) &&
    std::isfinite(d.x.B), "finite state");
23            expect(d.node_delta == e.node_delta && d.power_cap == e.power_cap && d.x.U == e.x.U, "deterministic");
24        }
25    }
26    std::printf(fails ? "SMOKE FAIL (%d)\n" : "SMOKE PASS\n", fails);
27    return fails ? 1 : 0;
28 }
29
```

cpp/tools/soak.cpp

47 lines SHA-256 b5ce02b4886ee59300b6ed4dc2d78a8668d8c2347af5451fb622038075efb400

```
1 // Soak test: N consecutive governor decisions on randomized observations spanning and exceeding the sensor
  ranges.
2 // Reports state bounds, non-finite values, resident memory before and after, and time per decision.
3 #include "omnicompass/governor.hpp"
4 #include <chrono>
5 #include <cmath>
6 #include <cstdio>
7 #include <cstdlib>
8 #include <fstream>
9 #include <random>
10 #include <string>
11 using namespace omnicompass;
12 static long rss_kb() { std::ifstream f("/proc/self/statm"); long a = 0, b = 0; f >> a >> b; return b * 4; }
13 int main(int argc, char** argv) {
14     const long long N = argc > 1 ? std::atoll(argv[1]) : 1000000LL;
15     const bool throughput = argc > 2 && std::string(argv[2]) == "throughput";
16     std::mt19937_64 r(20260923); std::uniform_real_distribution<double> u(-0.5, 2.5);
17     AllocationLaw law{};
18     if (throughput) {
19         law.backlog_release = true; law.power_ceiling = 9.0; law.power_ceiling_backlog = 9.0; law.spread_step = 0.0;
20         law.B_cap = 9.0;
21         law.size_at_full_cap = true; law.kI = 0.05; law.kE = 0.40; law.cap_min = 0.65; law.down_band = 1; law.
22         down_dwell = 10;
23         law.down_after_add = 3; law.kq = 0.30; law.U_gate = 0.45; law.push_release = 0.01;
24     }
25     Governor g{law}; Observation o; long nonfinite = 0; long rss_cp[4] = {0, 0, 0, 0};
26     double lo[5] = {1e300, 1e300, 1e300, 1e300, 1e300}, hi[5] = {-1e300, -1e300, -1e300, -1e300, -1e300};
27     double cap = 1.0; int nodes = 20; const long rss0 = rss_kb();
28     auto t0 = std::chrono::steady_clock::now();
29     for (long long i = 0; i < N; ++i) {
30         o.q = u(r); o.load = u(r); o.power = u(r); o.thermal = u(r); o.network = u(r); o.drift = u(r);
31         o.stale = u(r) > 1.0; o.security = u(r) > 2.3; o.conflicts = static_cast<int>(i % 7);
32         Directive d = g.step(o, cap, nodes);
33         cap = d.power_cap; nodes = std::max(8, std::min(40, nodes + d.node_delta));
34         const double v[5] = {d.x.E, d.x.U, d.x.I_U, d.x.S, d.x.B};
35         for (int k = 0; k < 5; ++k) { if (!std::isfinite(v[k])) ++nonfinite; lo[k] = std::min(lo[k], v[k]); hi[k] =
36         std::max(hi[k], v[k]); }
37         if (!std::isfinite(d.power_cap) || d.power_cap < 0.0 || d.power_cap > 1.0) ++nonfinite;
38         for (int c = 0; c < 4; ++c) if (i == (N / 4) * (c + 1) - 1) rss_cp[c] = rss_kb();
39     }
40     const double s = std::chrono::duration<double>(std::chrono::steady_clock::now() - t0).count();
41     std::printf("{\nmode\": \"%s\", \"rss_kb_at_25_50_75_100pct\": [%ld, %ld, %ld, %ld], \", throughput ?
42     \"throughput\" : \"power_protect\",
43     rss_cp[0], rss_cp[1], rss_cp[2], rss_cp[3]);
44     std::printf("\n\"decisions\": %lld, \"nonfinite_or_out_of_range\": %ld, \"rss_kb_start\": %ld, \"rss_kb_end\":
45     %ld, \"
46     \"ns_per_decision\": %.1f, \"years_at_5min_interval\": %.1f, \"
47     \"E\": [%ld, %ld], \"U\": [%ld, %ld], \"I_U\": [%ld, %ld], \"S\": [%ld, %ld], \"B\": [%ld,
48     %ld]}\n",
49     N, nonfinite, rss0, rss_kb(), 1e9 * s / N, N * 5.0 / 60 / 24 / 365.25,
50     lo[0], hi[0], lo[1], hi[1], lo[2], hi[2], lo[3], hi[3], lo[4], hi[4]);
51     return nonfinite == 0 ? 0 : 1;
52 }
```

cpp/scripts/compare_results.py

19 lines SHA-256 9f8b7ad299eeb5bdefca70e62796966c6efe6267a7c9d8f7fd1ad44ddb172da6

```
1 import csv,json,math,sys
2 from pathlib import Path
3 if len(sys.argv)!=4: raise SystemExit('usage: compare_results.py expected.csv actual.csv report.json')
4 e=list(csv.DictReader(open(sys.argv[1],newline='',encoding='utf-8'))); a=list(csv.DictReader(open(sys.argv[2],
5     newline='',encoding='utf-8')))
6 exact={'run_id','target_sign','born','convey','convey_start','convey_confirm','cert','cert_start','cert_complete',
7     'max_streak','actuator_saturated_periods'}
8 mis=[]; ma=mr=0.0
9 if len(e)!=len(a): mis.append({'kind':'row_count','expected':len(e),'actual':len(a)})
10 for i,(x,y) in enumerate(zip(e,a)):
11     for k,v in x.items():
12         if k in exact:
13             if v!=y.get(k):mis.append({'row':i,'column':k,'expected':v,'actual':y.get(k)})
14         else:
15             xv=float(v); yv=float(y[k]); ae=abs(xv-yv); re=ae/max(abs(xv),abs(yv),1e-300);ma=max(ma,ae);mr=max(mr,re)
16             if not (ae<=5e-12 or re<=5e-11):mis.append({'row':i,'column':k,'expected':xv,'actual':yv,'abs':ae,'rel':re})
17             if len(mis)>=100:break
18             if len(mis)>=100:break
19 r={'rows_expected':len(e),'rows_actual':len(a),'max_abs_error':ma,'max_rel_error':mr,'mismatches_capped':len(mis),
20     'pass':not mis,'tolerances':{'abs':5e-12,'rel':5e-11},'first_mismatches':mis[:20]}
21 Path(sys.argv[3]).write_text(json.dumps(r,indent=2)+'\n');print(json.dumps(r,indent=2));raise SystemExit(0 if r[
22     'pass'] else 1)
23
```